

ZaraiLink — Speaker Script

What to actually say for slides 4, 5, 6, 7

Group 5 | Umar, Fahad, Salman, Ibrahim

Total speaking budget: 10 minutes

How to read this. What's in quotes is roughly what you say. Square brackets like **[45s]** are timing cues. Italic notes like *point at the diagram* are stage directions. Don't memorise word-for-word — catch the rhythm and improvise.

1 Slide 4 — System Architecture

Skim before the talk — `zarailink_guide.pdf`

Priority	Pages	Why
Must	15–16	The layered diagram on page 15 mirrors the slide; §3.2 traces a request end-to-end.
Should	17	§3.3 auth flow + §3.4 token economy.
Should	18	§3.5 caching strategy + §3.6 data lineage.
Optional	1–3	§1.2 elevator pitch if you want a one-line opener.

Script (≈ 2 minutes 15 seconds)

[15s] *point at the slide.* “Okay so this is the architecture. Three layers I want to walk you through: the frontend on the left, the backend in the middle, and the data store on the right — plus a couple of third-party services.”

[30s] *point at the frontend block, then trace the arrow to Django.* “On the left, the user opens our web app in a browser. That's a React single-page app — charts come from Chart.js and Recharts, maps from React Simple Maps, and the smooth transitions are Framer Motion. The frontend talks to our backend over HTTPS, queries go in as JSON and results come back as JSON. The backend is Django REST Framework.”

[45s] *walk through the two boxes inside the backend.* “Inside the backend, two pieces matter. The Security and Access box verifies every request — it checks the user's token, looks up what they're entitled to see, and only then lets the request continue. The Intelligence Layer is where the actual product lives: a Search and Ranking module, a Supplier Aggregator that rolls up transactions per company, and an NLU Engine that figures out what the user meant. We'll unpack the NLU and ranking on the next slides.”

[20s] *point at the Postgres column.* “The data store is PostgreSQL. Everything lives there — accounts and token balances, subscriptions and access grants, the trade transactions and HS

codes, and the company directory with contacts and products. One database, four schemas, one source of truth.”

[25s] *point at Redis and OpenRouter on the right.* “Two outside dependencies. Redis is an NLU cache — when an identical query comes through, we skip re-running the ML models and serve the cached interpretation. If Redis is down, we fall back to an in-memory cache automatically, so it’s an accelerator, not a hard requirement. And OpenRouter is the language-model fallback — we only reach out to a hosted LLM when our regex price extraction fails on something tricky. About one query in twenty. The rest stay completely local.”

2 Slide 5 — Use Case and Dataset

Skim before the talk — `zarailink_guide.pdf`

Priority	Pages	Why
Must	64–65	§7.1 spreadsheet specs — exact row counts and columns.
Must	4	§1.6 user-facing tour — the seven pages map to the use cases.
Should	32	§4.5.1 the four-level product hierarchy.
Should	67–68	§7.5 ingestion scripts — how raw rows become DB rows.

Script (≈ 2 minutes 30 seconds)

[50s] *point at the use-case diagram.* “Two kinds of users on the platform. Regular users — importers, exporters, analysts — and admins.”

“Regular users can do four things. They can search the trade data, and they have multiple entry points: by HS code, by product name, or by typing a question in plain English. They can view a market-intelligence dashboard with charts. They can browse the company directory. And they can unlock premium contact information using tokens — discovery is free, but pulling up a phone number or email spends tokens from their balance.”

“Admins do two things — they manage user accounts, and they manage the trade data itself: ingestion, curation, HS code mapping.”

[80s] *this is the part that needs concrete numbers — slow down and hit each one.* “Now, the data. This is what makes the whole thing real instead of a toy.”

“Our industry partner is PAR. They handed us over nine thousand raw transaction records — sugar imports and exports spanning 2024 to 2025. Out of those nine thousand, we sat down and manually cleaned and standardized about five thousand. That meant fixing column headers, formatting HS codes to a uniform style, and fuzzy-matching company names that were spelled three different ways across different rows.”

“All of it goes into Postgres in a four-level hierarchy: Product, then Category, then Sub-Category, then Item. We didn’t invent this — it mirrors how the World Customs Organisation actually structures HS codes. Modeling our database the same way means analytics can roll up to whichever level the user is asking about, without us having to maintain separate tables for separate granularities.”

3 Slide 6 — ERD

Skim before the talk — zarailink_guide.pdf

Priority	Pages	Why
Must	88–90	Appendix C — model reference. Every table on the ERD with its key fields.
Must	32–33	§4.5.1–4.5.3 + Table 4.7 — Transaction model and product hierarchy.
Should	17	§3.4 token economy — explains the unlock tables.

Script (≈ 2 minutes)

Strategy. Don't read field names off the slide. The audience can see the ERD — give them the four-cluster shape and one interesting design choice, then move on.

[10s] *open with reassurance — the audience is staring at a wall of tables. “This looks like a lot, but it’s really just four groups of tables, and each group matches one part of the application.”*

[15s] *cluster 1 — keep brief. [accounts_user] “First group is identity. Just the user table. Email is the login field, and every user has a token balance integer right on their row — that’s the in-app currency for unlocking premium data.”*

[40s] *cluster 2 — the heart of the schema, this is where to spend the time. [trade_data_product, trade_data_productcategory, trade_data_productsubcategory, trade_data_productitem, trade_data_transaction, trade_data_productembedding] “Second group is the catalog. The four-level product hierarchy — Product, Category, Sub-Category, Item — each level pointing to its parent. And then the transaction table, which is the ledger every other table builds on. Every row is one shipment: when, what HS code, who bought, who sold, where from, where to. The search engine spends most of its time querying this table, so we’ve indexed every column the search filters by.”*

[20s] *cluster 3 — brief. [companies_company, companies_sector, companies_keycontact, companies_keycontactunlock] “Third group is the directory — companies, their sector, their key contacts, and a small junction table that records which user has unlocked which contact.”*

[25s] *cluster 4 — end with the one design-choice point worth volunteering. [subscriptions_subscriptionplan, subscriptions_usersubscription, subscriptions_useraccess] “Fourth group is the access side — subscription plans, user subscriptions, and a record of what each user has unlocked at the product level. You’ll notice we have two separate unlock tables — one for individual contacts, one for whole product categories. Different granularities of access, different lookups, so they’re kept separate.”*

[10s] *closer. “So that’s the schema in one breath: identity, catalog, directory, access. Each cluster is its own Django app, so changes in one don’t break the others.”*

4 Slide 7 — Search Pipeline

Skim before the talk — zarailink_guide.pdf

Priority	Pages	Why
Must	40–46	§5.1–5.4 walks the four stages: NLU, candidate retrieval, aggregation, ranking.
Must	47	§5.5 the ranking formula and how presets swap the weights.
Should	41	§5.2 SetFit + GLiNER — what each model is responsible for.
Should	43	§5.3 RapidFuzz typo correction + thresholds.

Script (≈ 2 minutes 15 seconds)

Strategy. Follow the arrows on the flowchart in order. Pause briefly at the diamond (Numeric?) — that branch is the one design decision worth calling out. Save the ranking detail for last so it lands.

[15s] point at the top of the flowchart. *“This is what actually happens when a user hits search. The input is a query string plus a scope — products, companies, or both. Everything below is the path that query takes.”*

[20s] point at the diamond. *“First branch is a numeric check. If the query is a number — say, four-digit HS code like 1701 — we skip the ML entirely and go straight to a direct database match. That’s a fast path for the analysts who know exactly what they’re looking for.”*

[45s] point at the NLU pipeline box — this is the ML-heavy slide. *“If it’s not numeric, the query goes through the NLU pipeline. Three pieces working together. SetFit handles intent — is the user buying, selling, or just researching. GLiNER pulls structured entities out of messy free text — product names, country names, quantities, price phrases. And RapidFuzz handles typos — if someone types “sugr” instead of “sugar”, we score it against our product catalog and accept matches above sixty percent. Together these turn an unstructured question into a structured query the rest of the pipeline can act on.”*

[20s] point at the validation box. *“Then validation. We check that the extracted entities make sense for the scope — if the user asked about products but we extracted a country, we disambiguate. The HS-code fast-path and the NLU path converge here, so everything downstream sees the same structured query shape.”*

[25s] point at the data layer, then branch left and right to access control and results. *“Then the data layer — one Django ORM aggregation query rolls every matching transaction up per supplier: average price, total volume, shipment count, recency. Results branch two ways. On the left, access control checks the user’s token balance and redacts premium fields they haven’t paid for. On the right, the surviving rows come back as supplier profiles with their product variants.”*

[20s] end on ranking — this is the one number worth volunteering. *“One thing worth knowing about the ranking step inside the data layer. We score every candidate with a hand-tuned blend of volume, shipment count, recency, and price — weights 0.4, 0.3, 0.2, 0.1 by default — and the weights swap based on what the user asked for. “Cheapest” sorts by price directly. “Most reliable” tilts toward shipment count. “Bulk” tilts toward volume. The default blend is what you see when no preference is signalled.”*

Time check

Slide	Budget	Where to spend the time
Slide 4	~2:15	Architecture walk left to right. Don't get into search internals — save that for slide 7.
Slide 5	~2:30	Use cases brisk. Dataset numbers slow — those are the academic credibility.
Slide 6	~2:00	Four clusters at a brisk pace. Audience already sees the ERD — give them shape, not field lists.
Slide 7	~2:15	Follow the arrows in order. Land the ranking weights at the end.
Buffer	~1:00	Transitions and audience questions.

One last thing. Practise once out loud with a stopwatch. The first time through is always 30 seconds longer than you planned. After that you'll know which sentences to drop if you're running long. The most droppable sentences: the Framer Motion / Chart.js name-check on slide 4, and the “two separate unlock tables” design note on slide 6.